

第二节 C++ 语法入门（上）

本课目标

学会定义变量，知道变量所对应的字节数。能够大概知道 `int` 类型的范围；
学会使用基本的算数运算符；
学会使用

本课讲解

Part 1：变量的定义。

变量的定义方式如下：

```
数据类型 变量名；
```

其中，常见的数据类型有 10 种：

变量类型			字节数	写法	占位符	表示范围
整形	短整型	有符号短整型	2	<code>short</code>	<code>%hd</code>	$[- 32768, 32767]$
		无符号短整型	2	<code>unsigned short</code>	<code>%hu</code>	$[0, 65535]$
	整形	有符号整型	4	<code>int</code>	<code>%d</code>	$[- 2147483648, 2147483647]$
		无符号整型	4	<code>unsigned int</code>	<code>%u</code>	$[0, 4294967296]$
	长整型	有符号长整型	8	<code>long long</code>	<code>%lld</code>	$[- 2^{63}, 2^{63} - 1]$
		无符号长整型	8	<code>unsigned long long</code>	<code>%llu</code>	$[0, 18446744073709551616]$
浮点数	单精度浮点数		4	<code>float</code>	<code>%f</code>	约 6 ~ 7 位有效数字
	双精度浮点数		8	<code>double</code>	<code>%lf</code>	约 15 ~ 16 位有效数字
	高精度浮点数		16	<code>long double</code>	<code>%Lf</code>	约 18 ~ 19 位有效数字
字符型			1	<code>char</code>	<code>%c</code>	$[- 128, 127]$ (对应 ASCII 码)
布尔型			1	<code>bool</code>	无	<code>true</code> 或 <code>false</code>

注：

- 1. 由于 `bool` 类型没有自带占位符（不在 `stdio.h` 头文件中），因此，我们通常使用 `%1d` 来替代（注意，是 `1`，不是 `l`！）；
- 2. 对于整形的所有有符号类型变量名称，可以在前面加上 `signed`。其中，`int` 可以被直接写成 `signed`。

Part 2：算数运算符与变量的使用

算术运算符中有两个单目运算符（正、负）以及五个双目运算符（乘法、除法、取模、加法、减法），其中单目运算符的优先级最高。

*tips：取模的意思就是计算两个整数相除得到的余数，即求余数。

单目与双目运算符

单目运算符（又称一元运算符）指被操作对象只有一个的运算符，而双目运算符（又称二元运算符）的被操作对象有两个。例如 `1 + 2` 中加号就是双目运算符，它有 `1` 和 `2` 两个被操作数。此外 C++ 中还有唯一的一个三目运算符 `?:`。

运算符	功能
<code>+</code> （单目）	正
<code>-</code> （单目）	负
<code>*</code> （双目）	乘法
<code>/</code>	除法
<code>%</code>	取模
<code>+</code> （双目）	加法
<code>-</code> （双目）	减法

举个例子

`a = b + c` 表示将 `b` 与 `c` 的和赋值给 `a`。

`d = d % e` 表示将 `d` 模 `e` 的值存在 `d` 中，即 `d` 模去 `e`。

算术运算中的类型转换

对于双目算术运算符，当参与运算的两个变量类型相同时，不发生类型转换，运算结果将会使用参与运算的变量的类型存储，否则会发生类型转换，以使两个变量的类型一致。

例如，对于一个整型（`int`）变量 `x` 和另一个双精度浮点型（`double`）类型变量 `y`：

- `x/3` 的结果将会是整型；
- `x/3.0` 的结果将会是双精度浮点型；
- `x/y` 的结果将会是双精度浮点型；

- `x*1/3` 的结果将会是整型；
- `x*1.0/3` 的结果将会是双精度浮点型。

[例题 1.2.1] A+B Problem

提交代码请点 [这里](#)

题目描述

输入两个整数 a, b ，输出它们的和 ($|a|, |b| \leq 10^9$)。

输入格式

两个以空格分开的整数。

输出格式

一个整数。

输入输出样例 #1

输入 #1

```
20 30
```

输出 #1

```
50
```

示例代码：

C 语言输入输出

```
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d", a + b);
    return 0;
}
```

C++ 输入输出

```
#include <iostream>
using namespace std;

int main()
{
    int a,b;
    cin >> a >> b;
    cout << a + b << endl;
    return 0;
}
```

*tip: 如果不知道输入输出怎么写, 请看 Part 3 中的内容。

自增/自减 运算符

有时我们需要让变量进行增加 1 (自增) 或者减少 1 (自减), 这时自增运算符 `++` 和自减运算符 `--` 就派上用场了。

自增/自减运算符可放在变量前或变量后面, 在变量前称为前缀, 在变量后称为后缀, 单独使用时前缀后缀无需特别区别, 如果需要用到表达式的值则需注意, 前缀自增/自减运算符的结果是运算后的结果, 后缀自增/自减运算符的结果将为运算前的值。

复合赋值运算符

复合赋值运算符实际上是表达式的缩写形式。可分为复合算术运算符 `+=`、`-=`、`*=`、`/=`、`%=` 和复合位运算符 `&=`、`|=`、`^=`、`<<=`、`>>=`。

举个例子

`d = d % e` 可以写作 `d %= e`

Part 3 : 基本输入输出

C++ 输入输出

在前面的学习中, 我们有用到 `cout` 语句进行输出, 这里我们接着学习与其相似的 `cin` 语句 (输入)。

就拿例题 1.2.1 中的代码来讲吧。

```
#include <iostream>
using namespace std;

int main()
{
    int a,b;
    cin >> a >> b;
    cout << a + b << endl;
    return 0;
}
```

这里，看到我们使用了两个与 `cin`、`cout` 有关的句子，他们分别是：

```
cin >> a >> b;
```

当程序执行这个语句时，我们输入了 `a`，`b` 两个变量的值。注意一下我们的写法，是：

```
cin >> 一个变量 >> 另一个变量 >> 又一个变量 >> .....;
```

接着看下一句话：

```
cout << a + b << endl;
```

当程序执行这个语句时，我们输出了表达式 `a + b` 的值。注意一下我们的写法，在这里，它是：

```
cout << 一个表达式 << 另一个表达式 << 其他的表达式 << .....;
```

可要看仔细了，`cin` 与 `cout` 语句中，对不同表达式的连接方式并不一样。对于 `cin`，我们使用 `>>` 符号进行连接，对于 `cout`，我们使用 `<<` 符号进行连接。

C 语言输入输出

在前面一道例题中，我也提供了使用 C 语言输入输出的程序，我们也来看一看。

```
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d", a + b);
    return 0;
}
```

Q1: `stdio.h` 头文件是什么？

A1: `stdio.h` 头文件中，包含了对方法（或者说，函数）`scanf()` 与 `printf()` 的定义。虽然在 C++ 中，`iostream` 头文件已经包含了 `scanf()` 与 `printf()`，所以在使用 `iostream` 头文件时，我们并不需要再引用 `stdio.h` 头文件以使用 `scanf()` 与 `printf()`。

我们仔细看一看这两句话。

```
scanf("%d%d", &a, &b);
```

我们把它分解一下：

1. `scanf()`;

这一部分引用了 `scanf()` 函数。

2. `"%d%d"`

这一部分是函数的第一个参数，表示需要读入的内容，观察一下发现，引号内部的内容是两个 `%d` 我们在哪里提过 `%d`？没错，它就是我们的占位符。占位符表示的是读取的下一个数据的数据类型。由于我们在输入时，`a`、`b` 两个数是以空格分开的，所以这里也可以写作：

```
"%d %d"
```

表示在读入两个整数时会以空格隔开。不过在读入数据不是字符串或字符时，我们可以使用空格分开，也可以使用换行符 `\n`。

3. `&a, &b`

这一句话表示的意思是，按照占位符的顺序，与其一一对应的变量。不过，这里要指出变量的地址，不然，`scanf()` 就不会把读入的数据写到你的变量里。

Q2：地址是什么？

A2：在计算机的内存中（注意！不是存储！）每一个内存空间都会有一个地址。这些地址的作用是指向内存空间以在存储和使用的时候，可以正确的将内容“放”在地址所对应的内存空间中。所以，地址的作用很大。它能帮我们找到我们需要的内容所在的位置，并交给 CPU 来读取该位置的内存中的数据。

对于第二个部分，它内部的字符可以是无止境的。不过要注意的是，一定要与后面的（第三部分中的）变量一一对应，不然程序会运行错误。同时也要注意，与第二部分不同的是，第三部分中的所有变量以逗号分隔开，第二与第三部分以逗号连接。

[例题 1.2.2] 判断下列 `scanf()` 语句书写的正误

- `scanf("%d", a);`
- `scanf("%d\nd%d%d", &a, &b, &c, &d);`
- `scanf("%d%d%d", &a, &b, &c);`
- `scanf("%lf %lf", &k, &j)`
- `scanf("%d %d %d", &a, &b, &c, &d);`
- `scanf("%c", &ch)`